Part 3: Automation framework

This document outlines the technical decisions and architecture implemented in the Playwright automation framework for the ParaBank application.

The scenario under tests simulates a customer log in, who performs a transfer, and later on check this movement on account details.

This scenario got a bit more complicated as I progressed with the framework, since I noticed Parabank deletes registered users every now and then. Therefore, it was not reliable create an account, and store the account info for login in a data json.

To bypass this, I implemented in the fixture a user register, storing it for later use, as a before hook before the test.

1. Environment Management and Configuration

The framework is designed to be scalable for different environments. It uses a hierarchical configuration approach:

- **Base Configuration**: playwright-base.config.ts contains shared settings such as timeouts, browser engines, and reporting.
- **Environment-Specific Configs**: Files like playwright-dev.config.ts inherit from the base configuration but overrides other parameters.

2. Security and Secret Management

To ensure security and prevent sensitive data leaks:

- **GitHub Secrets**: For pipeline execution, passwords are stored as encrypted secrets in the GitHub repository.
- **Environment Variables**: The framework utilizes .env files (per environment) to inject these secrets during execution.

3. Custom scripts

Custom scripts were added to package.json to simplify the interaction with the framework:

- **Execution**: Dedicated commands for different environments (e.g., npm run pw:dev:ui).
- **Maintenance**: A single command (npm run format) integrates both Prettier and ESLint.

4. Data Strategy with Faker

In order to generate data, I decided to use faker to make it random.

5. CI/CD Pipeline and Reporting

GitHub Actions workflow has been implemented for reporting visibility:

- **Automated Triggers**: Tests run automatically on push or pull request.
- **Artifact Management**: Test results are uploaded as artifacts.
- **GitHub Pages Integration**: The framework automatically deploys the Playwright HTML report to a public URL after each execution, providing immediate visibility of test results to stakeholders without requiring local setup.